4. 前缀和

前缀和与差分的核心思想是**预处理**，可以在暴力枚举的过程中，快速给出查询的结果，从而优化时间复杂度。

是经典的**用空间替换时间**的做法。

4.1 一维前缀和

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】前缀和
>
> **难度系数：** ★

【题目描述】

> 给定一个长度为n的数组 $a_1, a_2, ..., a_n$
>
> 接下来有 q 次查询, 每次查询有两个参数 l, r
>
> 对于每个询问, 请输出 $a_l + a_{l+1} + ... + a_r$

【输入描述】

> 第一行包含两个整数 n 和 q
>
> 第二行包含 n 个整数, 表示 $a_1, a_2, ..., a_n$
>
> 接下来 q 行,每行包含两个整数 l 和 r
>
> - $1 \le n, q \le 10^5$
> - $-10^9 \le a[i] \le 10^9$
> - $1 \le l \le r \le n$

【输出描述】

> 输出 q 行, 每行代表一次查询的结果

【示例一】

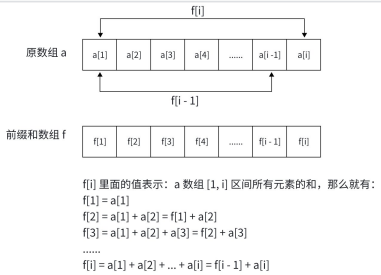
> 输入:
>
> ```
> 3 2
> 1 2 4
> 1 2
> 2 3
> ```
>
> 输出:

```
3
6
```

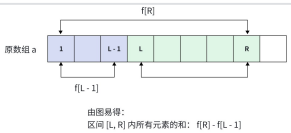
【解法】

前缀和模板题，直接套用「公式」创建前缀和数组，然后利用前缀和数组的「性质」处理 q 次查询。

1. 创建前缀和数组： $f[i] = f[i-1] + a[i]$



2. 查询 $[l, r]$ 区间和： $f[r] - f[l-1]$



【参考代码】

```
代码块
```

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

int n, q;
LL a[N];
LL f[N]; // 前缀和数组

int main()
{
    cin >> n >> q;

    for(int i = 1; i <= n; i++) cin >> a[i];

    // 处理前缀和数组
    for(int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] + a[i];
    }

    // 处理 q 次询问
    while(q--)
    {
        int l, r; cin >> l >> r;
        cout << f[r] - f[l - 1] << endl;
    }

    return 0;
}
```

4.2 最大子段和

> **题目来源：** 洛谷
>
> **题目链接：** P1115 最大子段和
>
> **难度系数：** ★★

【题目描述】

> 给出一个长度为 n 的序列 a ，选出其中连续且非空的一段使得这段和最大。

【输入描述】

> 第一行是一个整数，表示序列的长度 n 。
>
> 第二行有 n 个整数，第 i 个整数表示序列的第 i 个数字 a_i 。
>
> - 对于 40% 的数据，保证 $n \leq 1 \times 10^3$ 。
> - 对于 100% 的数据，保证 $1 \leq n \leq 2 \times 10^5, -10^4 \leq a_i \leq 10^4$ 。

【输出描述】

> 输出一行一个整数表示答案。

【示例一】

> 输入：
>
> 7
>
> 2 -4 3 -1 2 -4 3
>
> 输出：
>
> 4

【解法】

关于这道题的解法有很多种，我们先介绍利用「前缀和」的解法。

考虑以 i 位置的元素 $a[i]$ 「为结尾」的最大子段和：

- 在求「区间和」时，相当于是用 $f[i]$ 减去 i 位置前面的某一个 $f[x]$ ；
- 如果想要「最大子段和」，也就是「最大区间和」，那么用 $f[i]$ 减掉一个「前驱最小值」即可。

因此，我们可以创建 a 数组的「前缀和」数组，然后在遍历前缀和数组的过程中，一边「更新前驱最小值」，一边「更新当前位置为结尾的最大子段和」。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 2e5 + 10;

int n;
LL f[N]; // 前缀和数组

```

```cpp
int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        LL x; cin >> x;
        f[i] = f[i - 1] + x;
    }

    LL ret = -1e20;
    LL prevmin = 0;

    for(int i = 1; i <= n; i++)
    {
        ret = max(ret, f[i] - prevmin);
        prevmin = min(prevmin, f[i]);
    }

    cout << ret << endl;

    return 0;
}
```

4.3 二维前缀和

题目来源： 牛客网

题目链接： 【模板】二维前缀和

难度系数： ★

【题目描述】

给你一个 n 行 m 列的矩阵 A ，下标从 1 开始。
接下来有 q 次查询，每次查询输入 4 个参数 x_1, y_1, x_2, y_2
请输出以 (x_1, y_1) 为左上角，(x_2, y_2) 为右下角的子矩阵的和，

【输入描述】

第一行包含三个整数 n, m, q
接下来 n 行，每行 m 个整数，代表矩阵的元素
接下来 q 行，每行 4 个整数 x_1, y_1, x_2, y_2 ，分别代表这次查询的参数

$1 \le n, m \le 1000$

$1 \le q \le 10^5$

> $-10^9 \leq a[i][j] \leq 10^9$
>
> $1 \leq x_1 \leq x_2 \leq n$
>
> $1 \leq y_1 \leq y_2 \leq m$

【输出描述】

> 输出 q 行，每行表示查询结果。

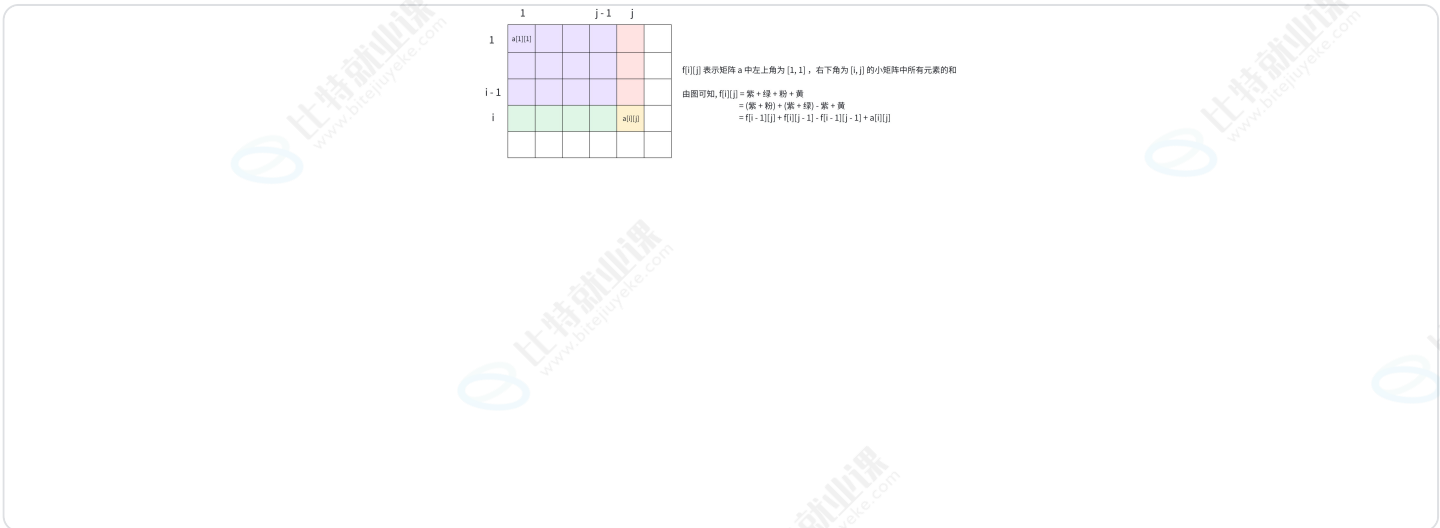
【示例一】

> 输入：
>
> 3 4 3
>
> 1 2 3 4
>
> 3 2 1 0
>
> 1 5 7 8
>
> 1 1 2 2
>
> 1 1 3 3
>
> 1 2 3 4
>
> 输出：
>
> 8
>
> 25
>
> 32

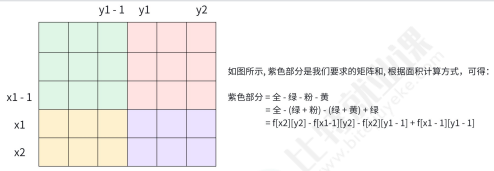
【解法】

二维前缀和模板题，直接套用「公式」创建前缀和矩阵，然后利用前缀和矩阵的「性质」处理 q 次询问。

1. 创建前缀和矩阵： $f[i][j] = f[i-1][j] + f[i][j-1] - f[i-1][j-1] + a[i][j]$



2. 查询以 (x_1, y_1) 为左上角，(x_2, y_2) 为右下角的子矩阵的和



【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1010;

int n, m, q;
LL f[N][N];

int main()
{
    cin >> n >> m >> q;
    // 预处理前缀和矩阵
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            LL x; cin >> x;
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + x;
        }
    }

    // 处理 q 次查询
    while(q--)
```

```
27      {
28          int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
29          cout << f[x2][y2] - f[x1 - 1][y2] - f[x2][y1 - 1] + f[x1 - 1][y1 - 1]
    << endl;
30      }
31
32      return 0;
33  }
34
35
```

4.4 激光炸弹

题目来源： 洛谷

题目链接： [HNOI2003]激光炸弹

难度系数： ★★

【题目描述】

一种新型的激光炸弹，可以摧毁一个边长为 R 的正方形内的所有的目标。

现在地图上有 $N(1 \leq N \leq 10000)$ 个目标，用整数 X_i，Y_i (其值在 $[0,5000]$)表示目标在地图上的位置，每个目标都有一个价值。

激光炸弹的投放是通过卫星定位的，但其有一个缺点，就是其爆破范围，即那个边长为 R 的正方形的边必须和 x,y 轴平行。

若目标位于爆破正方形的边上，该目标将不会被摧毁。

对于 100% 的数据，保证 $1 \leq n \leq 10^4, 0 \leq x_i, y_i \leq 5 \times 10^3, 1 \leq m \leq 5 \times 10^3, 1 \leq v_i \leq 100$ 。

【输入描述】

输入文件的第一行为正整数 N 和正整数 R，接下来的 N 行每行有 3 个正整数，分别表示 x_i, y_i, v_i 。

【输出描述】

输出文件仅有一个正整数，表示一颗炸弹最多能炸掉地图上总价值为多少的目标（结果不会超过 32767 ）。

【示例一】

输入：

```
2 1
```

```
001
111
```

输出：

```
1
```

【解法】

可以用一个二维矩阵将所有目标的价值存起来，其中 $a[i][j]$ 就表示 $[i,j]$ 位置的目标价值之和。

一颗炸弹能够获得的价值正好是一个 $R \times R$ 大小的一个正方形内所有目标的价值总和，那么我们可以求出 a 矩阵的前缀和矩阵，然后枚举所有边长为 R 的子正方形的价值之和，求出里面的最大值即可。

如何枚举边长为 R 的所有正方形：

- 仅需枚举右下角 $[x_2,y_2]$ $(R+1 \le x_2 \le 5000, R+1 \le y_2 \le 5000)$，那么结合边长 R 就可算出左上角 $[x_2-R+1, y_2-R+1]$。
- 代入前缀和矩阵中，就可以快速求出这个正方形内所有目标的总价值。

细节问题：

1. 题目中某一个位置会「重复」出现，因此 $a[i][j]+=w$；
2. 半径 R 可能「超过 5000」，此时炸弹可以摧毁所有目标，也就是整个矩阵的目标价值之和。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 5010;

int n, m;
int a[N][N];
int f[N][N]; // 前缀和矩阵

int main()
{
    cin >> n >> m;

    while(n--)
```

```
    {
        int x, y, v; cin >> x >> y >> v;
        x++, y++; // 下标从 1 开始计数
        a[x][y] += v; // 同一个位置有可能有多个目标
    }

    n = 5001;
    // 预处理前缀和矩阵
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + a[i][j];
        }
    }

    int ret = 0;
    m = min(m, n); // 如果 m 很大，相当于就是把整个区域全部摧毁
    // 枚举所有边长为 m 的正方形
    for(int x2 = m; x2 <= n; x2++)
    {
        for(int y2 = m; y2 <= n; y2++)
        {
            int x1 = x2 - m + 1, y1 = y2 - m + 1;
            ret = max(ret, f[x2][y2] - f[x1 - 1][y2] - f[x2][y1 - 1] + f[x1 - 1][y1 - 1]);
        }
    }

    cout << ret << endl;

    return 0;
}
```

5. 差分

前缀和与差分的核心思想是**预处理**，可以在暴力枚举的过程中，快速给出查询的结果，从而优化时间复杂度。

是经典的**用空间替换时间**的做法。

学完差分之后，大家会发现，**前缀和与差分是一对互逆的运算**。